COMPUTER NETWORKS · 2026

DTN

Delay/Disruption Tolerant Networking — A network built to survive delay and disruption

Team presentation

15 min

00 — FRAMING THE PROBLEM

When your text won't send in an elevator

Does that message disappear, or does it wait?

Computer Networks · 2026

00 — FRAMING THE PROBLEM

And the extreme case — why is there no "internet" for a Mars rover?

Why space? It's the most extreme case — crack it and disaster zones, deep sea, and remote areas follow. And it's where DTN is already built and flown (NASA JPL · CCSDS).

ROADMAP — THE JOURNEY OF A SINGLE PIECE OF DATA

Today's map: how one bundle travels from Earth to Mars

Stage	Question	The answering technology
① Problem	Why does TCP/IP break	long delay · disruption
② Store & carry	What if the path won't open	`Store-Carry-Forward`
③ Responsibility	Who is responsible end to end	`Custody`
④ Transport	What carries the space hop	`LTP`
⑤ Path	Which route, and when	`CGR` / opportunistic routing

The stage number at the top of each slide marks "you are here." At the end, our own demo confirms ①–⑤ all at once.

01 · STAGE ① PROBLEM — BACKGROUND

Extreme / long delay & disruption

- **Long, variable delay** — Earth–Mars one-way ~ `3–22 min` (RTT ~ `6–44 min`); the short, stable RTT TCP expects is simply impossible [1]
- **Intermittent connectivity → network partitions** — orbiting/moving nodes drop links often; a full end-to-end path may never exist at any single instant
- **Link asymmetry** — uplink and downlink capacities differ sharply
- **High physical loss** — bit errors from cosmic radiation / RF interference — not congestion

> Key point: no guarantee that "the entire path is alive at the same time." And it's not just space — disaster zones, deep sea, and remote areas share the same constraints.

02 · STAGE ① PROBLEM — PROBLEM DEFINITION

Where TCP/IP breaks — its 4 assumptions collapse

TCP/IP assumes...	...but on Mars
① A continuous end-to-end path	Paths never open all at once → even the 3-way handshake can't complete
② Short, stable RTT (timely ACK)	Minute-scale RTT → ACKs too late to grow the congestion window (BDP balloons) → poor utilization
③ Timeout means loss → retransmit	Late ACKs re-send data that was actually fine → wastes scarce link bandwidth
④ Loss means congestion → slow down	A radiation bit-flip is misread as congestion → TCP backs off exactly when it shouldn't

The irony: in space TCP slows itself down for the wrong reasons → throughput collapses. We don't need a faster TCP — we need a different premise.

03 · STAGE ②③ STORE, CARRY, RESPONSIBILITY — CORE IDEA

"Why not just store it and send later?" — fixing that naive answer step by step

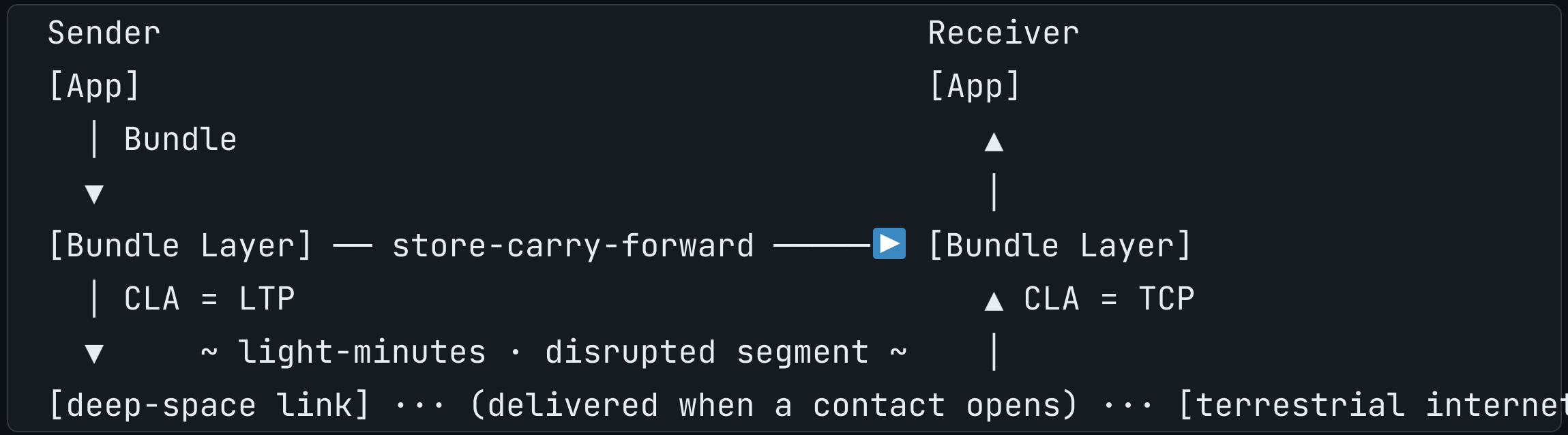
The naive answer — if the path won't open, a node holds the data and sends it when it does → this is `Store-Carry-Forward`. The whole path doesn't have to open at once.

But the gap: "If the node holding the data dies, who is responsible?" → `Custody Transfer` — the receiving node assumes delivery responsibility and guarantees retransmission (the handoff of responsibility in registered mail).

- Everyday analogy: a courier depot relay — once the next depot signs off on the parcel, responsibility is theirs from that point. The key that separates this from plain storage

04 · STAGE ②③ STORE, CARRY, RESPONSIBILITY — ARCHITECTURE

Bundle Protocol — one overlay layer



- Bundle Protocol v7 = `RFC 9171` (2022) [2] · architecture `RFC 4838` [3]
- Heterogeneous underlying networks are joined via a Convergence Layer Adapter (CLA) — TCP / LTP per segment

05 · STAGE ④ TRANSPORT — RELIABLE TRANSPORT

LTP — transport built for long-delay links

- Licklider Transmission Protocol = `RFC 5326` [4]
- Splits data into red (reliable · retransmitted) / green (unreliable)
- Defers ACK handling to match RTT → works even under minute-scale delay
- Used as the CLA for Bundle Protocol over deep-space links

> If TCP is a "conversation," LTP is "fire a long stream in one direction and settle up later."

MID-POINT RECAP — SO WHAT

So far: filling the gaps gives us a network that "arrives even when disconnected"

- Stage ① Problem — the continuous-path, short-RTT assumptions collapse
- Stage ② `Store-Carry-Forward` — don't wait for the whole path to open
- Stage ③ `Custody` — responsibility carries on even if the holding node dies
- Stage ④ `LTP` — reliable transport on the space segment by deferring ACKs

> So what's the big deal: data eventually arrives even if an end-to-end connection is never open all at once. Only one question remains — "which route, and when?"

06 · STAGE ⑤ PATH — ROUTING

Routing that knows the future vs. routing that bets on encounters

Approach	Environment	Idea	Cost
`CGR` (Contact Graph Routing)	Space (deterministic)	Precompute when each link will open	High compute · needs a contact plan
`Epidemic`	Opportunistic / mobile	Replicate to everyone you meet	High delivery · high overhead
`PRoPHET`	Opportunistic	Probabilistic delivery from encounter history	Medium
`Spray-and-Wait`	Opportunistic	Bound the replica count to stay in control	Low overhead

07 · STAGE ⑤ PATH — IMPLEMENTATIONS & CASES

It's already running in space

- `ION` (NASA JPL) — operational-grade implementation with built-in CGR · LTP · BP [5] · plus DTN2 · µD3TN · HDTN [10]
- `DINET` (2008) — first in-space DTN demonstration, using the Deep Impact spacecraft
- `ISS` — runs a DTN node on the International Space Station [11]
- `Psyche / DSOC` (2023~) — deep-space optical communications (physical layer) demonstration; complementary to DTN (upper = BP, lower = optical link) [6]
- `LunaNet / Artemis` — DTN in the lunar communications infrastructure standards
- Terrestrial — flood-prevention IoT via LEO constellations [8], remote/rural internet via train data-mules [9], underwater acoustic, sensor nets (ZebraNet), military tactical

08 — DIFFERENTIATOR ① (HERE WE LEAVE THE TEXTBOOK PATH)

We didn't just read about it — we ran it ourselves

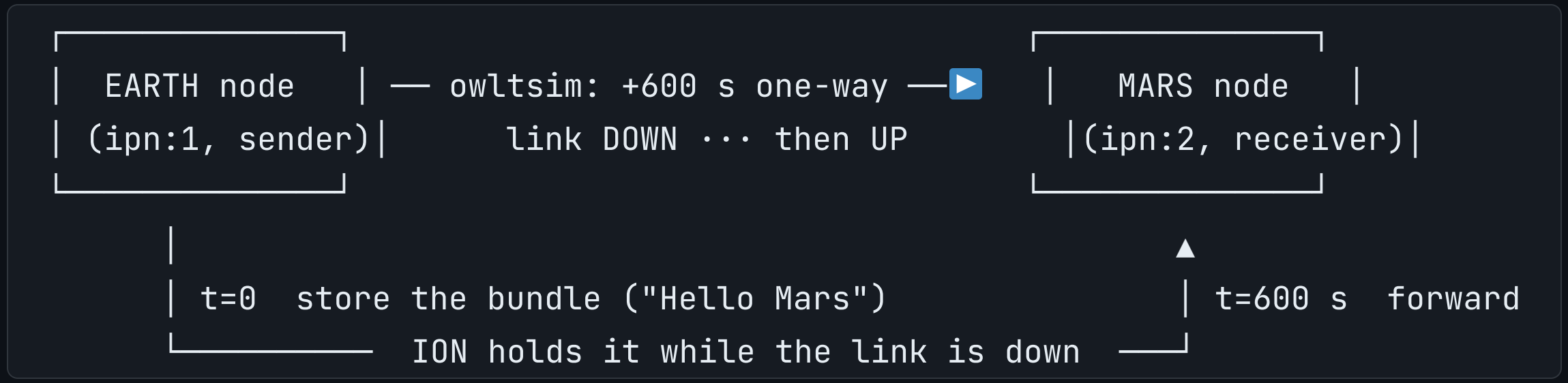
The plan: install the **same software NASA flies**, fake the Mars delay, and watch one message survive a dead link

DEMO (recording secured) ION-DTN · NASA JPL

What is ION? ION-DTN is NASA JPL's open-source DTN implementation — the same lineage that flies in space — published at `github.com/nasa-jpl/ION-DTN`. Open source means anyone can reproduce this — including us, on a laptop.

08 · DIFFERENTIATOR ① — THE SETUP IN ONE PICTURE

Our whole test, drawn: two nodes, one fake Mars delay



- **Two nodes** — smallest setup that still shows an end-to-end hop
- **One injected delay** — owltsim adds a `600 s` one-way light-time (more on it next)
- **Store → forward** — link down, bundle not lost: ION holds it, forwards when it can = Stages ②③ live

08 · DIFFERENTIATOR ① — WHAT OWLTSIM IS (AND WHAT IT IS NOT)

owltsim — the "Mars delay" knob, honestly bounded

- **What it is** — ION's One-Way Light Time simulator: it holds every signal between the two nodes for the configured delay (600 s) before letting it through. "Mars is light-minutes away" becomes a number we dial in on a laptop.
- **Models timing, not physics** — the delay length and the link going down → back up are real; the RF channel, antenna power, and radiation bit-errors are not modeled.
- **Not simulated at all** — `Store-Carry-Forward` and `custody` run in ION's real code; owltsim only supplies the delay, while ION genuinely stores, holds, and hands off the bundle.

> **The honest line:** the timing is real, the channel is faked — and we tell you exactly which is which.

08 · STAGE ②③ STORE, CARRY, RESPONSIBILITY — DIFFERENTIATOR ①, RUN & RESULTS

The run: send into a dead link, and it still arrives — 600 s later

Setup — two nodes up, 600 s delay on, receiver (`bpsink`) **first**, then send:

```
ionstart -I host1.rc              # sending node
ionstart -I host2.rc              # receiving node
owltsim  owlt.config              # inject one-way light-time delay (owlt=600s)
bpsink   ipn:2.1 &                # receiver first! (order matters)
bpsource ipn:2.1 "Hello Mars"     # send the bundle
```

Result — under TCP this dies at `t=0`; under DTN it's stored, then delivered at 600 s:

```
[t=0.0s]    "Hello Mars" stored at intermediate node (link down)
[t=600.1s]  light-time elapsed → forwarded → received "Hello Mars" (custody OK)
```


`custody OK` = the receiver **signed for it** — delivered intact across a dead link.

09 · STAGE ①④ — DIFFERENTIATOR ②, QUANTITATIVE COMPARISON

TCP vs DTN, by the numbers

Earth→Mars (one-way 12 min assumed)	TCP/IP	DTN (BP+LTP)
Connection setup	1.5 RTT ≈ 36 min (12 min × 3 one-way trips); if disrupted in between, session dies	No connection needed
On link disruption	Session drops · restart from scratch	Store and resume
Transport model	End-to-end, real time	Per-hop, asynchronous
Demo result (owltsim)	Timeout — delivery fails	100% delivery after delay

Assumptions: RTT = 2 × 12 min = 24 min; 3-way handshake = 1.5 RTT ≈ 36 min (when one-way is 12 min).

09 — ASIDE: WHY BDP EXPLODES

Bandwidth-delay product (BDP) — too much data "in flight"

- BDP = bandwidth × RTT. e.g. `1 Mbps × 1440 s (24 min round trip) ≈ 180 MB` simultaneously "in transit"
- TCP's window can't keep this much un-ACKed in flight → window-based flow & congestion control becomes meaningless
- DTN stores per hop, so it never puts everything "in flight" at once

10 — DIFFERENTIATOR ③, DECISION TREE

Critically — limitations and "when not to use it"

- Storage burden — intermediate nodes hold bundles for long periods and in bulk → buffer and discard-policy problems
- Overhead — bundle headers · fragmentation/reassembly complexity
- Immature congestion control · difficult security key management (BPSec `RFC 9172`) [7]

```
No continuous-path guarantee  +  large delay/disruption   →   DTN
Otherwise (stable, low-latency like the terrestrial internet)  →   TCP/IP (DTN is a ne
```

> A good presentation doesn't claim it's "a cure-all" — it knows when not to use it.

11 — WRAP-UP (RECOVERING STAGES ①-⑤)

In one sentence

> If the internet assumes "a path that is open now,"
> DTN assumes "a path that will open someday."

- What we did — started from the elevator scene and followed ① Problem → ② Store & carry → ③ Responsibility → ④ Transport → ⑤ Path, then verified it by running ION-DTN ourselves
- Core insight — thanks to `Store-Carry-Forward` + `Custody`, data arrives even when the path is never open all at once; not "it's slow" but "the premise is different"
- Looking ahead — operational in space (ION · ISS · LunaNet), with ongoing research extending to terrestrial and 6G non-terrestrial networks

Thank you — questions welcome

REFERENCES

Sources

- [1] NASA Mars Fact Sheet / JPL — Earth–Mars one-way light-time (~3–22 min)
- [2] `RFC 9171` — Bundle Protocol Version 7 (IETF, 2022)
- [3] `RFC 4838` — Delay-Tolerant Networking Architecture (IRTF)
- [4] `RFC 5326` — Licklider Transmission Protocol (LTP)
- [5] NASA JPL — Interplanetary Overlay Network (ION-DTN), open-source · github.com/nasa-jpl/ION-DTN
- [6] NASA — Psyche / Deep Space Optical Communications (DSOC), 2023~
- [7] `RFC 9172` — Bundle Protocol Security (BPSec)
- [8] Marin-de-Yzaguirre et al. — DTN for IoT LEO constellations (flood prevention) · Acta Astronautica 225 (2024)
- [9] E. M. Husni — DTN internet for remote areas via train systems · Proc. IEEE ICON (2011)
- [10] Douglass et al. — Fountain Code for High-Rate DTN (HDTN) · IEEE Access 11 (2023)
- [11] K. Schauer — Delay/Disruption Tolerant Networking Overview · NASA (2023)